Appendix K — How to disassemble

This is the how-to. It is the procedure this project uses, and it is the reasoning under that procedure: how a listing decides that a byte is an instruction, how a function gets a name, how a name is not yet a purpose, and how a purpose is not yet a description of the operation. Ghidra is the tool. The theories are about the work, and they apply the next time the tool is something else.

Date: 16 October 2026

Appendix I is how to check the C against the bytes. This appendix is how the project file gets to a C listing you can check, and how you decide what a function is before you are willing to name it. A menu path that was not written down in the record is not invented here. A theory is marked as a theory. A number that was counted is marked as counted.

Three jobs are always in front of you, and they are not the same job.

Job	The question	What a wrong answer looks like
Name	What will you type the next time you search?	A plausible word on a function you have not read.
Purpose	What question does this function answer for a caller?	The name of the one API it happens to call.
Operation	What does the body do, in order?	The C, trusted because it compiled into C.

A rename script does the first job, badly, if you let it do the other two. A decompiler does the third job, partially, and will happily invent a sentence for the second. The record's grades apply to all three. A pattern is not Known. A body that was read is Known as an operation. A purpose is Thought until a second caller, or a stop in a debugger, agrees with the sentence.

K.1 What is loaded

`toolbookunpacker` produces an honest NE image. It clears the Optloader self-load flag. It does not patch the entry table. That patch is an optional post-step beside the unpacker.

Stock Ghidra mis-reads two fields in that table. The entry-table segment byte was treated as signed, and `ne_cbenttab` was ignored. `TB80BAS` is the image, on the current set, with entry segment bytes at or above 128. The other modules do not need the post-step for that reason. An honest BAS image loads in the patched Ghidra 12.1.4 build. The unpacker does not rewrite those bytes. The Ghidra patch does.

Do not "fix" the image so that a stock Ghidra will open it by rewriting the entry table on disk and then treating that rewritten file as the program. The post-step is for the loader. The bytes of the functions are the unpacked image.

The packed originals are the wrong file to disassemble. On those files every segment but the stub has the relocation flag clear. The unpacked images are the ones with the relocating segments. A beautiful listing of the packed file is a listing of the wrong bytes. Chapter 20 counted this for the module-reference table. The same rule applies to every later listing.

Thought. A signed 8-bit read of a segment number turns 128 and above into a negative value. A loader that then rejects a non-positive segment drops those entries. They never become places to start disassembly. `ne_cbenttab` is the length of the entry table in the NE header. A loader that does not use that length is not walking the table the header describes. Both defects remove seeds. They do not alter a single opcode. That is why the patch belongs on the loader and not on the image.

K.2 Three addresses, and none of them is optional

Ghidra's `CodeN` is not the NE segment number. Segment 1 is `1000`, segment 2 is `1008`, and the usual step is 8, with gaps. `FUN_1160_03de` is offset `03DE` in the Ghidra segment whose base is `1160`. That base is not a live selector.

What you are holding	What it is
`maps/bas-ne-to-ghidra.md`	TB80BAS NE segment to Ghidra space. Use this one.
`maps/rtm-ne-to-ghidra.md`	TB80RTM, the same kind of table.
`maps/wdw-to-ghidra.md`	One boot's selectors. Dead after the next boot. Match the offset. Remap the selector.
`maps/html-segment-index.md`	Bases from one HTML export. If it disagrees with the three maps above, those three win.

A symbol-map line `1160:03DE` is the same pair. SoftICE and WDW do not load that file. The `.map` dumps are unsifted decompiler names. They were taken out of this edition. They stay on disk under `TBdecompile/20261015/20261012/`.

Theory: an address is a triple, or it is not an address. File segment, Ghidra base, live selector. Write all three when you have them, or write the one that is stable and say which one it is. The offset inside the NE segment is the stable half. The selector moves every boot. The Ghidra base is an invention of the loader, useful only inside that project file.

Mixing them is how a note becomes unverifiable. "The function at `4D27:2838`" is a live selector from one boot. "The function at `1028:0466`" may be a Ghidra pair. They are not comparable until the map says the Ghidra base that `4D27` was that day. Chapter 7 is the same rule for the cipher segment that a SoftICE module list had not loaded. The code was not absent. The segment was not mapped.

When you copy an address out of a Calls window, copy the offset and then do the map. Do not promote a selector into a `FUN_` name. `FUN_1028_0466` means the Ghidra space. If you did not do the map, you do not have that name.

K.3 What Ghidra is actually doing

Ghidra does not read the file from the first byte to the last and turn each byte into an instruction. That other method is a linear sweep. A linear sweep is what you get from a tool that assumes the whole range is code, in order, with no gaps and no data mixed in. It is the wrong default for a Win16 segment. A segment is a bag. It holds functions, jump tables, padding, and sometimes an immediate that happens to look like a prologue.

Ghidra's disassembler is a work list. The loader plants seeds. A seed is an address where an instruction is allowed to start. The entry point is a seed. Each exported entry the loader understood is a seed. The start of a code block is sometimes a seed. Everything else waits.

The disassembler takes a seed and asks the processor specification what instruction begins there. The specification for x86 is a SLEIGH file. SLEIGH matches the bytes against constructor patterns and, for each match, emits both the assembly text and a short program in pcode. Pcode is a small language of loads, stores, copies, adds, branches, calls, and returns. The assembly line is for you. The pcode is for the decompiler. They are produced together. They are not two independent opinions.

Once the instruction exists, the disassembler asks where execution can go next, and it puts those addresses on the work list.

The instruction	What is added to the work list
An ordinary instruction	The next address. This is fall-through.
A conditional jump	The target, and the fall-through.
A direct jump	The target only.
A direct call	The target, as a place a function may start. Fall-through of the call is also code, in the caller.

The instruction	What is added to the work list
A return, a return-far, or a halt	Nothing.
An indirect jump or an indirect call	Nothing, unless some other analyzer later proves a target.

The last row is the whole of the undisassembled-run problem. Recursive descent never walks into a function that nothing points at with a direct target. A Win16 program is full of those functions. They are reached by a far pointer built in two registers, by a function pointer stored in an object, by an export the loader dropped, or by a call that goes through a register. The bytes sit in the segment. The HTML prints `??`. The function is not missing from the program. It is missing from the work list.

This is the sentence to keep. The end of auto-analysis means the work list and the analyzer queue are empty. It does not mean every byte was considered. An empty queue is the analyzer having no further theory it is willing to act on. Undefined bytes are the bytes no theory claimed. Treat them as unread, not as data, until you have a positive reason they are data.

A positive reason is one of: a relocation or a reference that uses the address as a pointer to bytes, a table whose stride you have checked, a string the program actually loads, or a run of zeros used as alignment between two functions you already trust. "It was still undefined when analysis finished" is not a positive reason.

K.4 Why the walk stops, in more detail

A return stops the walk because the specification says the instruction does not fall through. That is correct for a real `RET` (`C3`) or `RETF` (`CB`). It is also how a mistaken decode hides the next function. If the byte before a real function is decoded as a return, the next function is never reached by fall-through. It will be reached only if something else names it. The prologue pass in K.6 exists because of this. It treats the byte after a real return as a candidate, and only as a candidate.

An indirect call stops the walk of the callee, not of the caller. The caller continues after the call. The callee is whoever was in the register or in the memory operand. Ghidra will not guess. You should not guess either, not from the call site alone. The call site tells you a pointer was used. The body, once you have found it, tells you what ran. Chapter 20 is the example already in the record: `CDBGETVALUEEX` locks, calls the code pointer in the `0x76` slot, and unlocks. The decompiler can show the call. It cannot show you which function sat in the slot on a given page. That is a runtime fact. A name for the slot's callee, chosen from the call site, would be a fiction.

A computed jump stops both directions if you are unlucky. The bytes after the jump may be a table of offsets, not code. If you force those bytes to disassemble, you manufacture instructions out of offsets, and those fake instructions fall into the real cases and split them. The practice is: when an indirect jump's targets are a dense table of words, define the table as words before you disassemble past it. This project has not needed that step to name the functions in the record. The rule is here so the next hole is not "fixed" by sweeping it.

Overlapping decodes are a different failure. x86 instructions are not a fixed size. The same bytes can be an instruction starting at address A and a different instruction starting at address A+1. Ghidra commits to one start. A seed that lands one byte off will consume the real instruction as raw material and produce a function that almost makes sense. The check is the one in Appendix I: the bytes, not the C. A function whose first instruction is not a prologue you recognize, and whose callers do not actually target that address, is a candidate for clearing and re-seeding one byte away. Clearing is destructive. Do it to the one function. Do not clear the segment.

K.5 Seeds, and the entry table as a theory of the program

The entry table is the program's own list of seeds. Each entry is an ordinal, a segment, an offset, and a flag that says whether the entry is exported by name. The resident and nonresident name tables are the program's own list of names for some of those seeds. An export name is the strongest name you will ever have. The linker published it. Another module's import table may cite it. You do not rename an export because a pattern found a better word. You may add a plate comment. The export name stays.

Internal functions have no such gift. They are reached by calls, and the calls are the only publication. A function with no call and no export is unpublished. It may still be real. It may be a candidate the prologue pass invented. Those two possibilities are why a created function is not yet a confirmed function.

Theory: the import tables are a purpose statement written by the linker. RTM's imports from BAS are `CDB*`, `VALUE*`, `STACK*`, `HEAP*` / `TBHEAP*`, and `RESMGR*`, plus `OBJBOOKVERSION`. That list is what RTM was willing to ask BAS to do. It is not a list of what BAS contains. BAS contains the `OBJ*` layer, and RTM does not link it except for that one version call. A theory of purpose that starts from "what do the other modules import?" will find the book API and will miss the object file. Both facts are in chapter 20. The method is the general one: read the import table as the cross-module purpose, and do not expect it to name the helpers inside the module.

Theory: an export that no import names is not dead. The EXE may call it by ordinal. An author tool may call it. The module may call it itself. `CDBSENDCREATE` is the case already settled. It is an export. It is the author path. A lesson open does not call it. "Present in the export table" and "on the open path" are different sentences. The first is settled by the NE file. The second is settled by a trace, or by reading the callers that the trace names.

Count the seeds after the load. The number of exported entries you can parse from the NE file, by hand or by the unpacker, should match the number of entry symbols Ghidra created. A shortfall on BAS, before the patch, is the signed-segment defect. A shortfall after the patch is a new defect, and it should stop the session. Do not begin naming in a project whose entry table is short. You would be naming a program with the doors closed.

K.6 Holes

A hole is a run of undefined bytes between two instructions, or between an instruction and the end of a block. Holes are the unread part of the listing. The HTML export of a hole is `??`. A search that only reads the HTML will report the hole as absent. Chapter 10 already says that absence from the HTML is not evidence.

Holes have a few shapes, and the shape is the first question, before any name.

Shape	Likely meaning	What to do
A few bytes, `90` or `CC` or zeros, between a `RET` and a prologue	Alignment	Leave them. Do not make them a function.
A run that starts with `C8` (`ENTER`)	A function the work list never reached	The first forced pass, below.
A run that starts with `55 8B EC` or the export prologue	The same, for a compiler that does not use `ENTER`	The second forced pass.
A run of word-sized values, dense, after an indirect jump	A jump table	Define words. Do not disassemble.
A run that contains the bytes `14 00` as an immediate, a `MOV CX, 0x14`, or a displacement	Not a record parser	Chapter 4. Thirteen such runs were counted. None of them is the far-pointer compare that builds an object.
A long run with no prologue and no table	Unread	Leave it unnamed. Forcing it because it is long manufactures code.

The `14 00 00` search is the cautionary case for this whole appendix. The bytes were found. They were found in the wrong instruction class. A theory that says "the object is three words of `0x14`" will light up every immediate, every short move, and every jump that happens to encode that displacement. The disassembler is not on your side here. It will decode those bytes as whatever the seed made them. Your job is to ask which instruction contains them. An immediate inside a call is a call. A displacement is a branch. A compare against a far pointer is the thing the theory was

about. Only the last one would have mattered, and it was not in the image that was searched.

K.7 Forcing the listing to finish

Two Script Manager passes force the listing to finish. Run them in this order. They do not name anything, and they do not touch code that is already defined.

1 The `C8` pass. Walk undefined bytes that are `C8` (`ENTER`) and create a function there.
2 The prologue pass. Walk undefined bytes. Create a function at `55 8B EC`, at the Win16 export prologue `45 55 8B EC 1E 8E D8`, at the byte after a real `C3` or `CB`, and at every NE entry that has no function. Accept a real `RET` or `RETF` before treating the following byte as code. Create functions only at entry-like symbols.

Run the `C8` pass first. The prologue pass is what runs second. Reversing them defines the wrong spans.

`ENTER` (`C8 iw 00`, in the common form) is the instruction a compiler emits for "make a frame of this many bytes." It is rare as the first byte of a data table and not impossible. The pass can be wrong. The protection is the rule that already exists: do not redefine a function that already exists, and do not treat the new function as named. After the pass, a new function that has no `RET` or `RETF` before the next defined range is a suspect. Clear that one. Do not clear the pass.

Theory: why the order matters. The prologue pass is greedy about the byte after a return. If it runs first, it can start a function one return too early and swallow an `ENTER` that belonged to the next function. That `ENTER` is then an interior byte. The `C8` pass, running second, sees a defined byte and walks past it. The function boundary is wrong, and the second pass cannot repair it because of the rule against redefining. Running `C8` first plants the seeds that are the most specific. The prologue pass then fills only what is still undefined. Specific seeds first, general seeds second. That is the same discipline as the entry table itself: trust the program's own list before you trust a pattern.

Thought, on the export prologue. `45 55 8B EC 1E 8E D8` is `INC BP`, `PUSH BP`, `MOV BP, SP`, `PUSH DS`, `MOV DS, AX`. Windows stack walking has long treated a saved `BP` whose low bit is set as a far frame. `INC BP` before the push is what sets that bit. A function that begins this way was written to be entered from outside the module, or at least to look like one. It is a better seed than a bare `PUSH BP`. It is still only a seed. The purpose is not "exported helper" until an entry or a far call says so.

The near prologue `55 8B EC` is `PUSH BP`, `MOV BP, SP`. Every compiler emits it, including compilers emitting the body of a function that is nothing but a comparison. It is the weakest of the seeds. Use it because the alternative is a hole. Do not use it as a reason to name.

K.8 SLEIGH, pcode, and the C you are reading

The bytes do not become C. The bytes become an instruction. The instruction becomes pcode. The decompiler simplifies pcode and prints something that looks like C. Four representations, and only the first one is the program.

Representation	Who produces it	What it is allowed to be
Bytes	The unpacked image	The evidence.
Assembly	SLEIGH, from the bytes	A reading of one instruction. Wrong if the seed was off by one.
Pcode	SLEIGH, with the assembly	The effects of that instruction: loads, stores, arithmetic, branches.
C	The decompiler, from simplified pcode	A hypothesis about a whole function, under a set of type guesses.

Simplification is why the C does not match the assembly line for line, and why that mismatch is not automatically a bug. Copy propagation removes a register that was only a temporary. A test and a branch become `if`. A flag set by `CMP` is folded into the comparison, so the `CMP` itself may not appear as a statement. Dead-code removal drops an instruction the decompiler decided could not matter. When the decompiler is right, the C is shorter than the assembly and easier to read. When it is wrong, the C is shorter than the assembly and missing the only instruction that mattered.

Appendix I is the procedure for that fight. This appendix only adds the reason you will have the fight: the C is downstream of two guesses, the guess of where instructions start and the guess of the types of the values those instructions move. Fix the guess. Do not edit the C in the notes and leave the database alone. The next export will overwrite your correction with the same guess.

Theory: a consistent lie is one cause. `SENDMESSAGE` was mangled at 321 sites in one RTM listing, and `LSTRLEN` at 282. The same nonsense at every site is not 321 separate misreadings. It is one signature, or one pointer size, applied everywhere the analyzer propagated it. The response is to stop, fix the type or the calling convention, and decompile again. The response is not to "correct" the arguments in the notes, site by site, while the database still holds the bad signature. The next person will export HTML and undo the corrections without knowing they were corrections.

Far pointers are the usual cause on this image. A far pointer is two words, segment and offset. A type system that wants one pointer will glue them, split them, or leave both as `undefined2`. The listings are full of `undefined2` and of values carried as a pair of words. That is what an untyped 16-bit value looks like after decompilation. It is not, by itself, evidence of a structure. A structure is a layout you have checked against more than one allocation site. Chapter 4's `0x76` and chapter 19's heap header are structures because the allocation and the field writes agree. Two `undefined2` parameters are not a structure. They are a far pointer the decompiler declined to name.

K.9 The string magnet

Ghidra finds runs of ASCII and defines them as strings. A later analyzer looks at immediates and, when an immediate equals the address of something already defined, it plants a reference. The decompiler then prints the name of that something instead of the number.

This is often what you want. A call that passes the address of an error string should show the string. It is also how a number becomes a fiction. The Ghidra address space for these modules is flat: segment bases at `1000`, `1008`, and so on, eight apart. An offset in one segment can be numerically equal to the address of a string that lives at that offset in a low segment. The analyzer does not know about the segment the instruction was using. It knows the number. The number matched. The reference is planted.

Chapter 4 already has the case. The string arithmetic in one decompilation is Ghidra matching constants against `s_CTbkSimpleOLEDispatch_CallMethod`. It is not a string compare in the program. The same magnet produced `s_DeleteBSTR` beside a call that is not `DeleteBSTR`. When a string name appears in a place that does not load a string, ignore the name and read the immediate. The immediate is the operand. The string is the analyzer's comment, printed in the slot where the operand should be.

Practice. If the instruction is `PUSH` of an immediate, and the called function is a string function, the magnet is probably right. If the instruction is an arithmetic compare, a function-pointer load, or a `GetProcAddress` ordinal that the decompiler rendered as a string, the magnet is a suspect. `GetProcAddress` in the `VBX*` wrappers takes an offset into a table of export names (`VbxInit`, `VbxNewBook`, and the rest). There the string is real, and it was checked against the file, not against the decompiler's label. That is the standard: a string claim is Known when the bytes at that file offset are those characters. A string claim that exists only as a Ghidra label is not yet a claim.

K.10 Theories of a name

A name is a key. It is not a paragraph. You will type it into a search box hundreds of times. A wrong key is worse than `FUN_`, because a search for the wrong key returns the wrong function with confidence, and a search for `FUN_` at least looks unfinished.

These are the ways a name can be earned. They are not interchangeable. A later way does not promote an earlier guess. It replaces it, or it fails to.

1. The binary named it. Export tables, import tables, and the strings passed to `GetProcAddress` when those strings have been checked in the file. This is Known. You do not improve `CDBFILEOPEN`. You can describe it.

2. The body matches an idiom you have already confirmed on another function. A GDI call sequence that is the same as `PaintRectangle`, a `GLOBALALLOC` of a size you have already tied to a structure, a `SENDMESSAGE` whose message number you have looked up. This is a candidate. On 11 October 2026, 1,918 of 10,991 functions were nameable this way, 17.5 percent. Confirmed renames after the later digs were 112. The gap is unreviewed candidates, not unknown programs. A candidate that has not been reviewed is not a name. The rename scripts in `TBdecompile/rename/` are what was applied. They rename only when the old name is still `FUN_…`. They do not invent the rest.

3. A unique constant selects it. An error code, a magic, a size that appears at one allocation and one set of field writes. `0x1F7A` is the out-of-indexes failure. `0xCCDD` is the heap magic in the model, because it was the magic in the reading. A constant that appears in fifty functions selects nothing. A constant that appears in one function, in a compare or a store and not as a displacement, is a reason to read that function. It is not yet a name.

4. The callers agree. Every caller you have read passes the same kind of argument and uses the result the same way. This is a theory of purpose leaking into the name. It becomes a name only after the body agrees. Callers lie by omission. They show you the use on one path.

5. A debugger stop put it on a path you care about. The function ran during open, or during paint, or during a script push. You now know a purpose on that path. You do not know the other paths. `TBHEAPCLEARCRITSECT` was a stop on the open path. It was not the last stop before the window. A name that says "the function that opens the book" would have been false. A note that says "it ran during open" is true.

6. A pattern matcher said so. The weakest source. Coverage by module, pattern only: RTM 32.7 percent, UTL 41.1, BMP 26, BAS 17.4, W16 17.3, LNL 12.2, CMP 5.6. CMP is low because most of it is the compiler, not a wrapper around a Windows API. A useful CMP name is `COMPILEREXPANDTEXT`. A name on a function that only moves a token is not. Naming another thousand `StrCpy` wrappers will not find the record reader. That was a counting goal in an 11 October note, not a research goal.

Theory: names should be boring. `PaintPolygon` is a good name if the body paints a polygon. `HandleObject` is a bad name even if the body handles an object, because the search box cannot tell it from the hundred other functions that also handle an object. Prefer the verb and the operand the body actually uses. Prefer the export's own stem (`CDB`, `OBJ`, `VALUE`, `RESMGR`, `TBX`, `VBX`) when the function is part of that family, so a search for the stem finds the family. Do not invent a new stem. The record has been careful about this: `RESMGR` type words are the words in the binary, not words chosen because they sound like resource kinds.

Theory: a thunk takes its name from the callee, never the other way around. The `TBX*` wrappers that are one call and a return exist so one calling convention can reach another. Their purpose is the call. Name them as the wrapper. Do not rename the callee to match the wrapper, or every internal caller of the real function changes meaning in your notes.

Theory: leave it unnamed. An unnamed function is easier to correct than a plausible wrong name. The discomfort of a `FUN_` in a chapter is useful. It marks the sentence as a sentence about an address. The moment the function is named, later chapters will treat the name as a word in the program. Chapter 11 had to undo a symbol-map line that called `FUN_11a8_04f6` by the export name `OBJFILEOPEN`. The export is a different offset. A wrong name in a map survives longer than a gap.

K.11 Theories of a purpose

Purpose is the smallest sentence that is still true of every caller you have read. It is not the nicest sentence that is true of the caller you like.

Role. Ask what the caller could not do if this function returned immediately. That is the role. If the answer is "it could not allocate," the role is allocation, even if the function also stores a magic. If the answer is "nothing, on this path," you have not found a purpose. You have found a call that might be a no-op, a probe, or a path you do not understand. Write that down as the state of the reading. Do not promote it to a verb.

Layer. Purpose lives in a layer. The layers in this program, as the imports and the bodies have sorted them, are roughly: the book API (`CDB*`), the object file (`OBJ*`), the script heap and the value stack (`HEAP*` / `TBHEAP*`, `STACK*`, `VALUE*`), the resource session (`RESMGR*`), paint, the script bytes, the VBX host (`VBX*`, which mostly calls `TB80VBX.DLL`), and the C++ edge (`TBX*`). A function that calls `BitBlt` and also allocates is a painter that allocates. Name and describe it in the paint layer. Do not file it under the heap because the heap call is the one you recognized.

Failure. The error a function stores is often a better purpose-seed than the success path. A function that returns null and stores `0x1F7A` exists, in part, to refuse a request the heap cannot satisfy. The success path is "here is a value." The failure path is "the index space is exhausted." Both belong in the sentence. A purpose written only from the success path will send the next reader looking for a parser when the interesting behavior is the refusal.

Frame. The frame size is a hint, not a name. `ENTER` of a handful of bytes, followed by one call, is a thunk or a setter. A frame of a few hundred bytes that calls no imported API is a walker of a buffer, or a parser, or a function with a large local table. You cannot tell those apart from the size. You can tell that it is not a one-line wrapper, so a wrapper-pattern name would be a lie. Use the frame to choose which functions are worth a long reading. Do not use it to title them.

Path. A debugger stop assigns a purpose on that path only. Open, paint, script, author, save. A function on the open path may also be on the save path and do a different half of the work. `CDBFILEOPEN` and `OBJFILEOPEN` are not the same function and not the same path into the file. Chapter 20. A purpose sentence should name the path it was observed on, or it should say "the body, regardless of path" and mean that the body was what was read.

The unpublished helper. Most of the program is unpublished. Helpers exist because the compiler split a function, or because a size check was shared, or because far calls are expensive and a near helper sits in the same segment. Their purpose is often "the three instructions the caller did not want to repeat." That is a real purpose. It does not deserve a grand name. A plate comment with the caller's name is enough until a second caller, in another subsystem, forces a real word.

Theory: purpose is stable under renaming, and renaming is not stable under purpose. You can change your mind about the purpose and the bytes do not care. You cannot change the purpose and keep the name without lying to your future search. When the purpose sentence changes, change the name in the same pass, or change the name back to `FUN_` until the sentence settles. A stale name is a purpose claim you no longer believe.

K.12 Theories of an operation

Operation is the ordered effect of the body. It is the only one of the three jobs that the bytes can settle by themselves.

Read it in this order. Skipping a step is how a helper gets the name of its caller.

1. Span. Where does the function start, and where does it return? One function, one span. A span that falls into the next prologue is two functions. A span that starts one byte late is not a function yet. Check the bytes at the start against the prologue you think you seeded.
2. Entry. Export ordinal, near call, far call, or a pointer. The entry decides the calling convention you are about to assume. A far export and a near helper do not use the stack the same way. If you assume one convention for both, the C arguments are a fiction from the first parameter on.
3. Saved registers. `BP` frame or not. `DS` reloaded or not. `SI` and `DI` saved or not. This tells you which values can outlive a call the function makes. It also tells you whether the export prologue ran.

4 Calls you already trust. List them by the names that are already Known. Ignore `FUN_` callees on the first pass, except to count them. A function is often explained by the three Known calls it makes and by nothing else.

5 Stores that outlive the call. A write to a field of an object, a store of an error code, a store through a pointer argument. These are the effects a caller can observe. Locals that die at the `RET` are not part of the purpose, though they may be the mechanism.

6 The return value. Which register, which flag, and whether the caller tests it. A function whose callers never test the return is not "returning the object" just because `AX` holds a pointer at the `RET`. Look at one caller.

7 Only then a sentence. One sentence of operation, graded Known if you read the body, and one sentence of purpose, graded Thought until something else agrees.

Theory: the operation is a trace through the body, not a summary of the name. Write the stores and the calls first, in order, and write the English second. If you write the English first you will select the instructions that match the English and call the rest compiler noise. Compiler noise is where the off-by-one lives. Chapter 19's heap is the example to imitate. The sentence came after the sizes, the tags, and the error codes. The tags were not invented to make the sentence round.

Theory: a decompiler artifact is an operation you must delete from the sentence. `*(uint *)0x0` in a listing is the decompiler losing a register. A string name that is not a string load is the magnet. An argument list that is identical nonsense at every call to one API is the bad signature. These may stay in the HTML. They do not stay in the chapter. The chapter says they are artifacts, or the chapter does not mention them.

K.13 The order of searches that does not lie

When you do not know what a function is, search in an order that cannot flatter you. Each step is allowed to promote a function to "worth reading." None of them is allowed to name it except the first.

1 Exports and imports. If it has a published name, stop. Read the body to describe it. Do not rename it.

2 Checked strings. File offsets you have dumped, not labels in the listing. A `GetProcAddress` name, an error string, a window class.

3 Unique constants, in the right instruction. Compare, store, allocate. Not a displacement. Not the immediate of an unrelated call.

4 Callers of a Known function. Who calls `VALUENEWLONG` tells you who builds a long. It does not tell you that those callers are parsers.

5 Callees of the function you are reading. Three Known callees are a hypothesis. Zero Known callees means you are in a desert. Read more carefully, or come back with a better seed. Do not name the desert after the nearest town.

6 Prologue shape. Near, far, `ENTER`, export prologue. This chooses the convention for the C. It does not choose the name.

7 Pattern lists. Last. And only as a queue of functions to review, never as a list of names to apply in bulk because the count looks low.

Reverse this order and the work feels faster. You will produce names. They will be the names of the patterns, which are the names of the Windows API, which are not the names of the program. UTL at 41.1 percent pattern-nameable is the trap. A high score means the module is a wrapper. Wrappers are the easy part. The reader, if it is anywhere, is in the part the patterns do not cover. CMP at 5.6 percent is the more honest number.

Theory: search for the failure first. On a function you suspect is interesting, find the store of a nonzero error or the early return. The success path is longer and full of moves. The failure path is short and says what the function refuses to do. Refusals are specific. Success is often "copy the arguments somewhere." The record reader, whatever it is, will refuse malformed input somewhere. A search that only looks for successful parses will find every copy loop in the module.

K.14 What not to name, and what not to conclude

Do not name a function because the neighborhood is named. Neighbors share a segment because of the linker, not because of a theme. A paint helper and a string helper can sit next to each other if they were compiled in the same file. The segment is not a chapter.

Do not name a function from one caller. The caller has a purpose. The callee may be shared.

Do not conclude that a function is absent because the HTML does not contain it. Force the listing, or say that you searched the HTML and not the image.

Do not conclude that a byte sequence is a structure because it appears in a hole. Ask the instruction class.

Do not conclude that a function runs on open because it is an export, or that it does not run because a trace you stopped early did not reach it. `TBHEAPCLEARCRITSECT` was not the last stop. A later stop does not erase the earlier one. An earlier stop does not crown a function.

Do not "correct" mangled arguments in prose while the database still has the bad type. Decompile again.

Do not disassemble the packed file and call it the program.

Do not mix a live selector with a Ghidra base in the same sentence without the map.

Do not treat a counting goal as a research goal. 1,918 candidates was a count. The reader was not going to be candidate number 1,919.

Ghidra has a function-ID library that hashes a body and proposes a name from a catalogue of known libraries. These modules are not in that catalogue. A hit, if you ever turn the analyzer on, is a coincidence of instructions, not a port of some other program's function into ToolBook. Leave function ID off, or treat a hit as a joke you can look at later. It is not a source of names. This is advice. The project did not need a hit from that library to do the readings in the record, and it should not acquire one.

K.15 Scripts, exports, and the HTML as a snapshot

A readable decompile is not one rename of the project. Each pass has a thing it is allowed to change.

Pass	What it may do	What it must not do
1. Finish the listing	The two passes in K.7	Name a function. Redefine a function that already exists.
2. Mark candidates	Note a body that matches a pattern	Treat the pattern as the name.
3. Confirm a name	Rename when the body matches, by hand or by a script in `TBdecompile/rename/`	Rename a function whose body does not match. Invent names for the functions that do not match.
4. Replace the reading copy	Export HTML again, with the date in the filename	Search yesterday's HTML for a name you applied today. That file still says `FUN_`.

The HTML export is a snapshot of the database, not the database. Searches you can repeat belong on the snapshot only if you name the snapshot. A claim that "the listing does not contain X" must say which file, which date, and whether the forced passes had been run. Otherwise the claim is about your export settings.

The scripts are idempotent on purpose. A second run finds no `FUN_` left to rename, so it does nothing. That is the property you want from any later script. A script that renames whatever is at an address, regardless of the current name, will overwrite a correction you made by hand and will do it quietly. Refuse that script. The address might have been wrong. The hand correction might have been the only careful reading in the segment.

Java and Python rename scripts live in `TBdecompile/rename/`. Copies of the nameable TSV files live under `TBWorldview/Modules/tb80*_knownnames/`. The TSV is the candidate list. It is not the script. Running the TSV as

if it were the script is how a pattern becomes a name without a review.

Ghidra `CodeN` is not the NE segment number. Use `TBdecompile/maps/` before typing a segment into a debugger. The same map is what a script should use if it is going to mention a segment in a comment. A script that writes `Code47` into a note has created a name that dies when the project is reloaded into a different address layout. Write the NE segment, or write the offset only.

K.16 One session

1. Unpack. Apply the entry-table post-step only for an image that needs it. BAS does. The others, on this set, do not.
2. Open the image in the patched Ghidra 12.1.4 build. Confirm it is the 16-bit language. A 32-bit load of a 16-bit segment will look busy and be nonsense from the first call.
3. Let analysis run. Do not treat the end of analysis as a finished listing. Count entry symbols against the NE entry table. Stop if the count is short.
4. Run the `C8` pass, then the prologue pass. Spot-check one new function for a real return. Clear the one that does not have one.
5. Check one export you already trust, such as `CDBFILEOPEN` (ordinal 8), against `maps/` before you trust the segment name on the screen.
6. If one imported API is mangled at every site, stop. Fix the signature. Decompile again. Do not start the naming pass on top of a bad convention.
7. Read one function by the order in K.12. Write the operation first. Write a purpose sentence second, and mark it as a theory.
8. Rename that one function, or run the rename scripts for a family you have already reviewed. Do not run a pattern list as if it were the script.
9. Export a new HTML if that is the copy you search. Keep the old file. It is the trail of what the listing used to say.
10. If the C and the bytes disagree, Appendix I wins. The decompiler does not get a vote against the bytes.

Come back when a chapter has used a `FUN_` name in a claim. The cleanup for that function is: confirm the span is a real function, read the body, and only then replace the name. Do not rename the neighborhood because the neighborhood is unnamed.

K.17 A method, applied to functions the record already settled

These are not new findings. They are the method above, run backwards on names that are already in the record, so the next unnamed function has a shape to imitate.

`CDBFILEOPEN`, ordinal 8. The binary named it. The job is description, not renaming. The description in chapter 20 is that it does not call `OBJFILEOPEN` by name. The decompile reaches the object file through unnamed helpers. The purpose of the export is "open," because that is what RTM links it for. The operation includes a path that is not a direct call. Both sentences can be true. A theory that said "the open export is the open routine, so it must be the one that reads the header" would have collapsed them, and would have sent the search to the wrong function.

`VALUENEWLONG`. The name is the binary's, or the family's. The operation is the construction of a long, a 4-byte block, tag `0xB`, on the script heap. The purpose is "make a script value that is a long." It is not "parse a record," even if a record parser would have to call it. The constant `0xB` is a tag, earned by being the second argument at the allocation, not by being a number that appears somewhere in the function. That is the difference between a unique constant used in the right instruction and a byte that merely occurs.

`VBXINIT`. The export name is a seed for a search, not the end of one. The body loads `TB80VBX.DLL` and calls `GetProcAddress` for `VbxInit`. The operation is a forwarder. The purpose is "start the VBX host that lives in the other

module." A theory that said "VBXINIT constructs a VBX object" would be wrong. The view object, `0x11C` bytes and the magic `0xABC1`, is `VBXNEWVIEW`, and that one is implemented in BAS. The method separates them because the bodies are different, not because the names share a stem. The stem made them neighbors in the search box. The bodies decided which one allocates.

The thing that is still `FUN_`. Apply the same steps and stop where the evidence stops. If you have a span and no trusted callee, you have an operation you have not finished, and you do not have a purpose. Leave the name. A chapter may cite the address. The appendix you are reading is the reason that citation is allowed to look unfinished.

K.18 How to read the names Ghidra already printed

Before you add a name, read the names the tool invented. They are a classification, not a caption. Each prefix is a theory the database is willing to state in public.

Prefix	What the database is claiming	What it is not claiming
`FUN_`	A function symbol at this Ghidra address. The digits are the base and the offset.	That anyone has read it. That the span is right.
`LAB_`	A branch target inside a function.	A function. Do not rename a `LAB_` as if it were a callee.
`DAT_`	A data symbol at an address.	A type. `DAT_` is "something lives here." The size is a separate guess.
`s_`	A string the ASCII pass defined, with a mangled prefix of the characters.	That any instruction loads it. The magnet may have glued it on later.
`switchD_`	A jump the decompiler decided was a switch.	That the jump table was checked by hand.
`unaff_`	A register used in the function and not written by the function. Incoming.	That it is junk. It is often a parameter the signature missed.
`extraout_`	A register a call wrote, which the declared return type does not include.	That the call failed to return. The type is short of the truth.
`in_`	An incoming register the convention accepted as a parameter.	That the convention is the one the compiler used.
`local_`	A stack slot.	That the source had a local variable with a meaning.
`param_`	A stack slot the decompiler thinks the caller passed.	That the caller pushed it. Check the call site.

`unaff_SS` and `unaff_CS` are the usual 16-bit artifacts. The stack segment and the code segment are real, and they are almost never "the argument." An `unaff_BX` or `unaff_DX` beside a call that does not push enough words is a different matter. Watcom and other compilers of this era could pass arguments in registers. This project has not established which compiler built which module, and this appendix will not invent one. The operation-level question does not need the vendor's name. Look at the call site. If the caller loads `AX` and `DX` and then calls, and the callee consumes `AX` and `DX` without reading them from the stack, those registers are the arguments. The decompiler will call them `unaff_` until the signature says otherwise. Promoting them to `param_` is a type edit, in the database, followed by a fresh decompile. Writing "the first argument is BX" in a note, while the HTML still says `unaff_BX`, is a correction that the next export will not remember.

`extraout_DX` next to a function that returns a far pointer is the high half of the pointer, sitting outside a return type that was declared as a single word. The fix is the return type, not a paragraph about `DX`. After the type is a far pointer, a new decompile should fold the pair. If it does not, the call is not returning a far pointer and the theory was wrong. That is a cheap experiment. Do it before the theory reaches a chapter.

`param_` slots that the caller never pushes are the mirror image. The decompiler assumed a stack convention and invented homes for values that arrived in registers, or it assumed too many arguments because an earlier signature said so. One bad signature on an imported API does this at every site. Section K.8 already said to stop and fix it. The prefix is how you notice. If every call to one import has the same extra `param_`, the import is the bug.

`local_` variables with an address of the form `local_4`, `local_6`, climbing by two, are the frame. Their number should be in the same neighborhood as the `ENTER` immediate or the `SUB SP, n` you can see in the bytes. If the C has twelve locals and the prologue subtracts four, the decompiler has invented stack slots out of registers. Trust the prologue. Appendix I.

K.19 What the analyzer queue is willing to believe

Auto-analysis is a queue of theories, each allowed to act, each ordered by a priority you do not see in the HTML. When the queue is empty, the tool stops. It does not mean the theories were true. It means none of them had a further action it was willing to take.

The theories, in the order they matter for this kind of image, are roughly these. The names here are the ideas, not a menu path. A menu path that was not written down is not invented.

Disassemble from seeds. The work list of K.3. Empties when every direct target has been turned into an instruction or has been refused.

Find strings. Runs of printable bytes become `s_` symbols. This runs early, which is why the magnet in K.9 has something to stick to. A string found in a code hole may be a real string that a function returns, or it may be bytes that happened to be printable. A string with no reference after analysis is a candidate for "not a string." Do not delete it on the first session. A later seed may be the function that loads it.

Create functions at call targets. A direct call is a strong seed. The callee gets a function symbol even if you never asked. This is why exports and internal callees acquire `FUN_` names without the two forced passes. It is also why a bad decode that contains a byte pattern looking like a direct call will spawn a function at a nonsense address. The child function is not evidence. The call instruction is the evidence. Check the bytes of the call.

Search for prologues in what is still undefined. Ghidra has its own version of this idea, and a more aggressive variant that the tool itself treats as dangerous because it false-positives. The `C8` pass and the prologue pass are a narrower form of the same idea: specific bytes, undefined ranges only, no redefinition. Prefer them to a switch that sweeps every hole. A sweep is how a jump table becomes code.

Recover switches. One analyzer asks the decompiler which jumps look like switches and then defines the tables. When it is right, a `switchD_` name appears and the cases become a body you can read. When it is wrong, a table of offsets has been consumed and the real cases are split. Treat `switchD_` as a theory. The check is the same as any table: the bytes are word-sized, dense, and in range for the segment. If they are not, clear the switch and define words.

Stack and types. Frame size, locals, parameters, then the propagation of any type you have applied. This is the pass that turns one corrected `SendMessage` into 321 readable call sites, or into 321 sites that are wrong in a new way. After a type change, this is the pass you are re-running, whether you think of it that way or not. Budget a fresh export. Do not compare the new C to your memory of the old C. Compare it to the bytes.

Constant references. The magnet. Last in spirit even when the priority number says otherwise, because it depends on strings and data existing already.

The queue will not do the following, and you should not expect it to.

It will not read a far pointer built in two registers and disassemble the target. It will not follow `GetProcAddress`. It will not apply NE relocations that the image has not already applied. It will not notice that two `FUN_` bodies are the same pattern of moves. It will not know that ordinal 8 is the open you care about. Those are your seeds, your maps, and your names.

Theory: a tool's refusal is information. Ghidra stops at an indirect call because it does not have a target. That refusal is more honest than a linear sweep, which would have invented a target by falling into whatever bytes came next. When the listing shows a hole after a return, the tool is saying "I will not pretend." The forced passes are you deciding, on a narrow pattern, to pretend a little. Keep the pretense narrow. The moment the pattern is "any undefined byte," you have thrown away the refusal that made the listing trustworthy.

K.20 Calling conventions are a theory of the stack

A calling convention is a contract between the call site and the body. The decompiler has to assume one in order to print parameters. If the assumption is wrong, every parameter in that function is wrong, and every sentence you write about "the first argument" is about a stack slot the caller may not have pushed.

On a Win16 module the contracts in play are at least these. Which one a given function uses is a question for the bytes, not for the vendor's manual.

Contract	What the call site does	What the return does	What the decompiler gets wrong if it assumes another
Near, caller cleans	Pushes words, then a near call. The caller adds to SP after the call.	RET with no immediate.	It will show the callee popping arguments the callee does not pop.
Near, callee cleans	Pushes words, then a near call. The caller does not adjust SP.	RET n.	It will show the arguments as locals of the caller, still on the stack.
Far, callee cleans	Pushes words, then a far call.	RETF n.	A near assumption splits one far return into a near return and a stray immediate.
Registers	Loads AX, DX, BX, CX and calls, with few or no pushes.	Values in AX or in AX:DX.	It prints unaff_ and invents param_ slots nobody pushed.
Far pointer return	The caller uses both AX and DX.	AX is the offset, DX is the segment, or the reverse. Check one caller before you pick.	It keeps extraout_DX forever if the return type is a single word.

The practice at one function is short. Look at three call sites if you have them, or at one if you have only one. Count the pushes between the previous call and this one. Count the RET immediate, in bytes, not in words: RET 4 pops four bytes, which is two near words. If the pushes and the RET immediate disagree, you are not looking at a simple contract. Someone passed extra words, or the decompiler combined two calls, or the span is wrong. Do not paper over a disagreement with a name.

RETF with no immediate means a far function whose caller cleans, or a far function with no stack arguments. Both are real. The export prologue of K.7 is a hint that the function expected to be entered from outside, which on Windows is often far and callee-cleaned. The hint is not the contract. The RET immediate is the contract.

Theory: fix the convention at the function, then at the import, and never in the prose. A custom convention on one function stays on that function. A convention on an imported API is stored with the external symbol, and every caller in the module inherits it. That is the lever behind the 321 identical SENDMESSAGE sites. It is also the lever behind a single internal function whose C is nonsense while its neighbors are fine. Change the smaller thing first. If one function is wrong, edit that function's signature. If every call to one import is wrong, edit the import. Then decompile again and read the bytes of one site before you trust the other three hundred.

K.21 Relocations, ordinals, and other numbers that are not yet addresses

A relocating segment is a segment the loader is prepared to patch. The NE relocation table lists places in the segment where a segment number, an offset, or an imported ordinal has to be filled in when the segment is mapped. The unpacked image is the right file to disassemble. It is not, by that fact alone, a file in which every immediate is the runtime address.

Before you treat a far target as a place you can open in the listing, know which of these you are holding.

What you see	What it might be
A far call whose target is a FUN_ you can click	A direct call the disassembler already followed. The seed worked.
A far immediate that is not a code address in this project	A fixup that was not applied, a segment number stored as an immediate, or the magnet.
A call through a register	Not an address. Do not invent the callee from the register's last store unless that store is a direct load of a known function.

What you see	What it might be
An import by ordinal	A function in KERNEL, USER, GDI, or another module. Appendix E is the ordinal list this manual carries for the three system libraries. Resolve it before you describe the call. "Calls KERNEL" is not an operation. "Calls `GETMODULEHANDLE`" is, once the ordinal says so.
`GetProcAddress` of a checked string	A runtime bind. The callee is the string, in the module that was loaded. `VbxInit` is the case already checked. The decompiler will not disassemble into `TB80VBX.DLL` because of that call. You open the other module.

Theory: a number in a listing is an address only after you have said which address space. Offset in the segment, Ghidra base plus offset, live selector plus offset, ordinal, file offset. Five spaces. The relocation table is how the program itself translates between some of them at load time. If you skip it, you will eventually add two numbers from different spaces and get a `FUN_` that is not in the file. The maps exist so that translation is written down once. Use them. Do not do the arithmetic in your head and then forget which side you started on.

Ordinals deserve a second warning. An import recorded as "KERNEL, ordinal 49" does not contain the letters of the function's name anywhere in the module. A string search will not find it. The only honest resolution is the ordinal table, checked against a source you trust, which for the system libraries is Appendix E of this manual. A function whose only interesting call is an unresolved ordinal is a function you have not started to read. Resolve the ordinal first. The body will be shorter afterwards, because the call will mean something and the moves around it will turn into setup for that call.

K.22 A segment is sometimes a compilation unit and usually is not

Chapter 20 counted 29 `OBJ*` exports. Twenty-six of them sit in one NE segment, Ghidra `1170`. That neighborhood means something, because the prefix was already a name and the linker piled one body of code into one segment. The inference ran from the names to the segment. It does not run backwards.

Theory: count the prefixes before you infer a theme. Take the exports of one segment, which are the names the binary is willing to defend. If they share a stem, the segment is probably one compilation unit or one class, and a helper in that segment is a candidate for the same layer. If the exports of the segment have three stems, the segment is a bag the linker filled, and a helper's neighbors tell you nothing. Internal `FUN_` symbols do not vote. They have no stem until you invent one, and inventing one in order to make the count come out is the fallacy the theory exists to block.

The practical consequence is where you spend a long reading. A segment whose exports are all `VALUE*` is a place to study values. A segment whose exports are mixed, with a hole in the middle, is a place to apply K.6 and K.7 and then leave. The hole is not "the rest of the value code" because it shares a segment with a value export. The hole is unread.

The same count is the right way to think about the C++ edge. `TBX*` names are a stem. A function with that stem that is one call and a return is a thunk, and its purpose is the callee. A function with that stem that has a real body is the place the work happens. The stem got you the list. The body length, the frame size, and the trusted callees sort the list. Sorting a stem by body length is a better afternoon than naming every `FUN_` in the segment the stem lives in.

K.23 Three witnesses, and which one is allowed to outvote

You will have, for any function you stay with, up to three witnesses. They disagree. The disagreement is the work.

The bytes, disassembled with a span you are willing to defend. This witness outvotes the other two on the question of what the function does. It cannot tell you the path that reaches the function on a Tuesday. It can tell you the function refuses a null pointer. If the bytes and the C disagree, keep the bytes. Appendix I is that rule written as a procedure.

The decompiler, which is a reader of pcode under type guesses. It outvotes your memory of a function you have not opened in a month. It does not outvote the bytes. It is allowed to suggest a structure. It is not allowed to settle one. Use

it to find the calls and the stores, then verify the calls and the stores against the assembly before they enter a sentence graded Known.

A debugger stop, SoftICE or the Watcom debugger, on a path you provoked. This witness outvotes the other two on the question of whether the function ran on that path. It does not outvote them on what the function does on a path you did not take. A stop during open is not a description of save. A function that did not stop during a short trace is not absent. The trace may have been stopped first. Chapter 19's note that `TBHEAPCLEARCRITSECT` was a stop, and not the last one, is the shape of an honest debugger sentence: it ran, here is where, and the window came later.

Write the witness into the sentence. "The body stores `0x1F7A`" is the bytes. "The C shows a call to `VALUENEWLONG`" is the decompiler, and it needs the bytes behind it before it is Known. "It was on the stack during open" is the debugger. A sentence that mixes them without saying which is which cannot be graded, and a claim that cannot be graded should not be in the record.

Theory: do not use one witness to decorate another. The bad habit is to take a debugger stop, find the nearest named function in the HTML, and describe that function's C as what ran. The stop gave you a selector and an offset. The map gives you a Ghidra address, if you do the map. The bytes at that address are what ran. The nearest named function may be the next export, a hundred bytes away, and it may be a function the stop did not reach. Distance in the listing is not proximity in time. The Calls window is the better debugger tool when you can get it, because it names the return address, which is a caller, which is a fact about the path. Even then, map the selector before you search.

K.24 False friends, functions that look like the thing you want

Every long disassembly grows a set of functions that look like the goal. This program already has a collection. They are listed here so the next one is recognized as a member of the collection, not as a discovery.

It looks like	It is	Why the resemblance happens
A parser of the page record	`VALUENEWLONG`, and the other `VALUENEW*` constructors	They allocate a typed value. A parser would call them. They are not the parser.
The open routine, one function	`CDBFILEOPEN` plus unnamed helpers plus `OBJFILEOPEN`	The export name says open. The operation is a chain.
The reader, because the bytes `14 00 00` are present	An immediate, a `MOV CX, 0x14`, or a displacement	The search did not ask the instruction class.
The last step before the window	Whichever heap call the trace had reached when you looked	The trace had not finished.
A VBX object constructor	`VBXINIT`	The name says init. The body is `GetProcAddress`. The allocator is `VBXNEWVIEW`.
A string compare	An arithmetic instruction the magnet decorated with `s_`	The immediate matched an address in a flat space.
The record reader, because it is unnamed and large	A compiler pass, a painter, or a thunk cluster	CMP is mostly unnamed because it is the compiler. Large and unnamed is the normal state of this image.

Theory: the goal-shaped function is usually the support act. The function you can find by a pattern is the function the pattern was built from. Patterns are built from Windows APIs and from the exports, because those are the names you already had. The function that implements a private format does not call a distinctive API. It moves bytes, compares a tag, and calls the support act. When a search returns a perfect match, you have found the support act. Start from its callers, not from its name. The callers that are still `FUN_` are the list. The callers that are exports are the door. The private function is the one in between that failed every pattern, which is why the pattern search did not return it.

This is not a claim that the record reader has been located by elimination. Elimination on 10,991 functions is not a method. It is a claim about where not to stop. Stopping at the first function that allocates, opens, or contains a familiar constant is how the support act gets the name of the feature.

K.25 Repeatability, the trail, and the database

A listing is a database with a date. The HTML is a print of it. The record cites the print, or it cites the bytes. It should say which.

The practices that make a later session able to check this one:

Keep the previous HTML. A claim that a name was absent is a claim about a file. If you overwrite the file, the claim cannot be checked, and it also cannot be distinguished from a claim about the program. The trail directories under `TBdecompile/` are that habit already. Do not tidy them into the manual. Do not delete them because the new export is prettier.

Date the filename of every export you intend to search. Searching an undated `TB80BAS.DLL.html` is how yesterday's `FUN_` and today's name get mixed in one sentence.

Prefer a script that refuses to rename a function whose name is no longer `FUN_`. The refusal is the audit. A script that always writes the name will erase a hand correction and leave no trace except the new name, which looks deliberate.

Do not clear a segment and re-disassemble it because one function is wrong. You will lose every type you applied and every span you forced, and the queue will refill the segment with the same theories as the first time, plus whatever the aggressive options happen to do. Clear the one function. If the bytes need to go back to undefined so the `C8` pass can see them, clear the code units of that span only, and know that you have done it.

Write plate comments for theories, and names for operations you have finished. A plate can say "thought: parser." A name `ParsePage` cannot be unread by a search. When the thought dies, the plate is edited and the search does not still find `ParsePage` in every chapter that copied it. This is the same rule as the record's grades. Thought is allowed to be wrong. A name is a kind of Known, whether you meant it to be or not, because later readers will treat it as one.

Theory: the database is not the program, and the manual is not the database. The program is the unpacked image. The database is one reading of it, with seeds and types and names layered on. The manual is the set of sentences that survived. When you are about to write a sentence, ask which of the three it is about. A sentence about the program can be checked against the bytes in a year, on a different machine, with a different Ghidra. A sentence about the database can be checked against the dated HTML. A sentence about the manual is a sentence about what you believed. All three are allowed. They are not interchangeable, and only the first one remains true if the database is lost.

K.26 A longer pass over one function, as a template

What follows uses `HEAPVALUETOPOINTER`, which chapter 19 already settled, as a template for the notes you take on a function that is not settled. The point is the shape of the note. The facts are the chapter's.

Span and entry. It is an export in the heap family, so the name is the binary's and the entry is a published one. You do not rename it. You do not look for a prologue in order to decide it is a function. The entry table already did that, once the loader was patched.

Operation, from the bytes the chapter records. The index is the low 10 bits of the value (`value & 0x3FF`), scaled by 2, and used to read a selector word in the heap. The offset is the other half of the value. WDW's first instructions on that export were `and si, 03FF` and `add si, si`, which is the debugger witness agreeing with the body. The sentence of operation is: a script value is not a pointer; it is an index and an offset, and this function turns them back into a pointer.

What the decompiler will do to it. It will be tempted to print the mask as a bitwise and, which in this case is faithful. It may also attach a string or a `DAT_` to one of the small constants if the magnet finds a match. `0x3FF` as a string would be the magnet, and you would ignore it. The `and` and the shift are the operation. They are simple enough that the C and the bytes should agree. If they do not, the span is wrong or the language size is wrong. That agreement, on a function

this small, is a good calibration of the project file before you trust it on a function that is not small.

Purpose, and the purpose it does not have. The purpose is "recover the address of a script value." The purpose is not "parse a record," not "allocate," and not "the inverse of every constructor." `HEAPPOINTERTOVALUE` is the other direction, and chapter 19 is careful that the low 10 bits of that direction were not a clean reading. A template note copies that caution instead of forcing the two functions to be perfect mirrors. When the reverse direction is muddy, say so. Do not tidy it so the pair looks designed.

What you would have done if the name had been `FUN_`. Mask and scale of a field is a unique-constant search (`0x3FF` on a value, in an `AND`, not as a displacement). The callers would be every function that holds a script value and needs a pointer, which is too many to name from. The purpose sentence would stay Thought until the heap chapter's other functions agreed: the constructors store the same shape the pointer function recovers. Agreement across several functions is what would promote it. One function with a mask is only a function with a mask.

Use that last paragraph as the bar. A single body can give you an operation. A purpose you are willing to put in the record should survive contact with a second function, or with a stop on a path. If it cannot, it is a plate comment.

K.27 Mistakes this listing has already paid for

They are collected so they are one list, not a set of surprises distributed through the chapters.

The packed file was the wrong image, because the relocation flag was clear on every segment but the stub. Disassemble the unpacked image.

Stock Ghidra dropped BAS entries whose segment byte was at or above 128, and it did not honor `ne_cbenttab`. The patch is on the loader. The image stays honest.

`CodeN` was treated as an NE segment number. It is not. The step is 8, with gaps, from `1000`.

A symbol-map line called a `FUN_` by an export's name. The export was a different offset. The map line is not the function.

The bytes `14 00 00` were found, in the wrong instruction class, thirteen times. The far-pointer compare was not among them.

`TBHEAPCLEARCRITSECT` was taken for the last thing before the window because it was the stop in hand. A later stop was `VALUEFREE`.

`CDBSENDCREATE` was taken for the open path because it is an export and its name says create. It is the author path.

The string `s_CTbkSimpleOLEDispatch_CallMethod` was taken for a compare. It was the magnet.

`SENDMESSAGE` and `LSTRLEN` were read site by site as if each mangled argument list were a separate fact. Each list was the same bad signature.

A pattern count was treated as progress toward the reader. 1,918 candidates did not move the reader.

None of these required a new kind of mistake. Each one was a witness used outside its authority: the wrong file, the wrong address space, the wrong instruction class, a stop that was not the last stop, an export that was not a path, a label that was not a load, a type that was not the program, a count that was not a finding. The theories in this appendix are the authorities written down. A new mistake will be a witness used outside them in a way the list has not named yet. When that happens, add the case. Do not add a name for it.

K.28 The bytes that move the work list

The work list is not a metaphor. It is the consequence of a handful of opcodes. When you are looking at a hole, or at a function that seems to end too soon, these are the bytes that caused it. The numbers are the instruction, not a Ghidra name.

Bytes	Instruction	What the work list does
`E8 iw`	Near call, relative	The caller falls through. The target is a new seed.
`9A ow sw`	Far call, absolute offset and segment	The caller falls through. The target is a seed only if that segment is in this image and the offset lands on code. A far call into another module is an import, not a seed in this file.
`FF /2`	Near call, indirect	The caller falls through. No callee is added.
`FF /3`	Far call, indirect	The same. This is the object slot and the function pointer.
`E9 iw`	Near jump, relative	No fall-through. The target is a seed.
`EB ib`	Short jump	The same, with a one-byte displacement. A displacement of `14` is one of the ways the bytes `14 00` appear without being a record.
`EA ow sw`	Far jump, absolute	No fall-through. Often a thunk into another segment.
`C3`	Near return	The walk stops.
`C2 iw`	Near return, pop `iw` bytes	The walk stops. `iw` is the contract of K.20.
`CB`	Far return	The walk stops.
`CA iw`	Far return, pop `iw` bytes	The walk stops. The export that forgot to be a seed often sits immediately after one of these.
`C8 iw 00`	`ENTER` of `iw` bytes, nesting level 0	Not a branch. A seed only if you plant one. The first forced pass.
`55 8B EC`	`PUSH BP` / `MOV BP, SP`	Not a branch. The weak prologue.
`45 55 8B EC 1E 8E D8`	The export prologue of K.7	Not a branch. A stronger seed, still not a name.

`/2` and `/3` are the ModR/M reg field, not a second opcode byte you can search for as the literal `02`. Searching the file for `FF 1E` or whatever the ModR/M happens to be will find some of the indirect calls and miss the rest. Do not do that search and call it a census of function pointers. Ask the disassembler, after the seeds are in, which calls have no target. That list is the list of unpublished callees. It is the closest thing this method has to a table of contents for the unread part of the module.

A far call's segment word is in the instruction. If that segment word is a fixup, the bytes in the file may not be the segment you will see at runtime. K.21. If the disassembler already turned the call into a `FUN_` in this project, the fixup was either applied or was unnecessary. If the call is still an immediate pair of words, you are looking at a number, not at a function, until the map or the relocation entry says otherwise.

Theory: the hole after a `CB` or a `CA` is the function the entry table was supposed to name. That is the ordinary shape of a Win16 segment: return, then the next function, with no call inside the segment pointing at it, because the callers are in other segments and the pointer was built at runtime or the call is far and the seed was dropped. The prologue pass that starts at the byte after a real return is doing this theory with a narrow pattern. When you look at such a hole by hand, do not ask "what calls this?" first. Nothing in the already-disassembled code may call it. Ask whether the bytes are a prologue. If they are, plant the seed. The callers will be found later, as far calls and as stops, or they will not be found. Absence of a near caller is not absence of a function.

K.29 When to stop reading

A function can be read forever. The decompiler will keep offering a local you have not explained. The rule for stopping:

You may stop when you can write the operation as the stores and the trusted calls, in order, and the bytes of those stores and calls match the C. You do not need to explain every `local_`. You need to explain every write that outlives the return, and every call whose name you are going to repeat in a chapter.

You should stop, and come back later, when the next fact you need is a convention you have not fixed, a segment you have not mapped, or an ordinal you have not resolved. More staring will not produce it. The sentence you write on the way out is "unread past the call to ordinal N" or "unread past the far immediate," which is a precise kind of unfinished. It is more useful than a guessed purpose, because the next session knows where to start.

You should stop immediately when you notice you are about to type a name in order to feel finished. That impulse is the 1,919th pattern. The function will still be at its address tomorrow. The wrong name will be in the search box until someone is brave enough to delete it.

K.30 Food for the next hole

A few questions, to ask of the next undefined run before anyone names it. They are questions, not findings.

Is this run undefined because the work list never reached it, or because a previous pass defined the bytes before it as data and stopped? The first is a missing seed. The second may be correct.

Does the first byte match `ENTER`, the near prologue, or the export prologue? If none of them, what positive reason is there to treat the run as code at all?

If you disassemble it, does a real return occur before the next thing you already trust? If not, the span is wrong or the run is not code.

Which of the three addresses are you about to write down? If you cannot say "NE segment" or "Ghidra base" or "selector from the boot of this date," do not write the number.

Which of the three jobs are you about to do? If you are about to invent a name in order to have a purpose, stop. Write the operation you can actually see, or write nothing.

Does the function store an error? Start there.

Does every call to one imported API in this hole look mangled in the same way as the rest of the module? Then the hole is not the problem. The signature is.

Does the interesting constant appear as a compare, a size, a store, or a displacement? Only the first three are about the constant. The fourth is about a branch.

Who imports this module, and do they import this entry? If they do not, the function can still be the one. It is not the one because of the import table. It is the one, if it is, because of the body.

Would a wrong name be worse than no name? It would. The search box has a long memory.

None of these questions names the record reader. They are the filter in front of the next claim that something is the record reader. A claim that survives the filter is a candidate for the record. A claim that skips the filter is a pattern, and this program has already produced 1,918 of those.